PYTHON ACTIVITY-BASED LEARNING

10 Real-World Case Studies

Algorithms | Flowcharts | Pseudocode | Python Programs
Test Cases and Outputs

STUDENT DETAILS

Student Name	D. Sai Pranathi
Roll Number	26000888
Branch	CSE (AI & ML)
Section	CSM 1
Course	B.Tech · First Year
Institution	Dr. RVR NRIIT DTBU

ACTIVITY OVERVIEW

This activity applies beginner-level Python programming to practical situations involving education, utilities, healthcare, payroll, retail, event planning, attendance, banking simulations, inventory, and student performance. Each case study includes a problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, sample output, test cases, and conclusion.

Note: All names, values, rates, and sample records are fictional and used for educational demonstration. Verify institutional or official rules before using any calculation in real life.

LIST OF SELECTED CASE STUDIES

The following 10 topics are covered in the order requested:

1. University Scholarship Allocation

2. Progressive Electricity Billing

3. Hospital Billing and Patient Classification

4. Employee Payroll and Deduction Analysis

5. Retail Billing with Multiple Discount Rules

6. Campus Event Financial Planning

7. Attendance Recovery Computation

8. ATM Transaction Sequence Simulation

9. Warehouse Inventory Reconciliation

10. Student Performance Trend Analysis

Document structure: Cover page → Topic list → Detailed case studies 1–10. Each case study is presented in a consistent format to make review and practical demonstration easier.

CASE STUDY 1: University Scholarship Allocation

1. Problem Statement

Develop a program that checks scholarship applicants using aggregate marks, subject-wise minimum marks, attendance, and family income. Eligible applicants are ranked by aggregate marks. If two applicants have equal marks, higher attendance is preferred; if still tied, the lower family income is preferred.

2. Objectives

• Check every eligibility condition consistently.

• Rank eligible applicants using clear tie-breaking rules.

• Display eligibility, rank, and scholarship recommendation.

3. Assumptions and Rules

• Marks are percentages from 0 to 100.

• Minimum aggregate is 75%; each subject must be at least 60%.

• Attendance must be at least 75%.

• Annual family income must not exceed ■2,50,000.

• The sample data represents fictional applicants.

4. Algorithm (Step by Step)

1. Start the program.

2. Store each applicant's name, aggregate, subject marks, attendance, and annual income.

3. Check aggregate, every subject mark, attendance, and income against the limits.

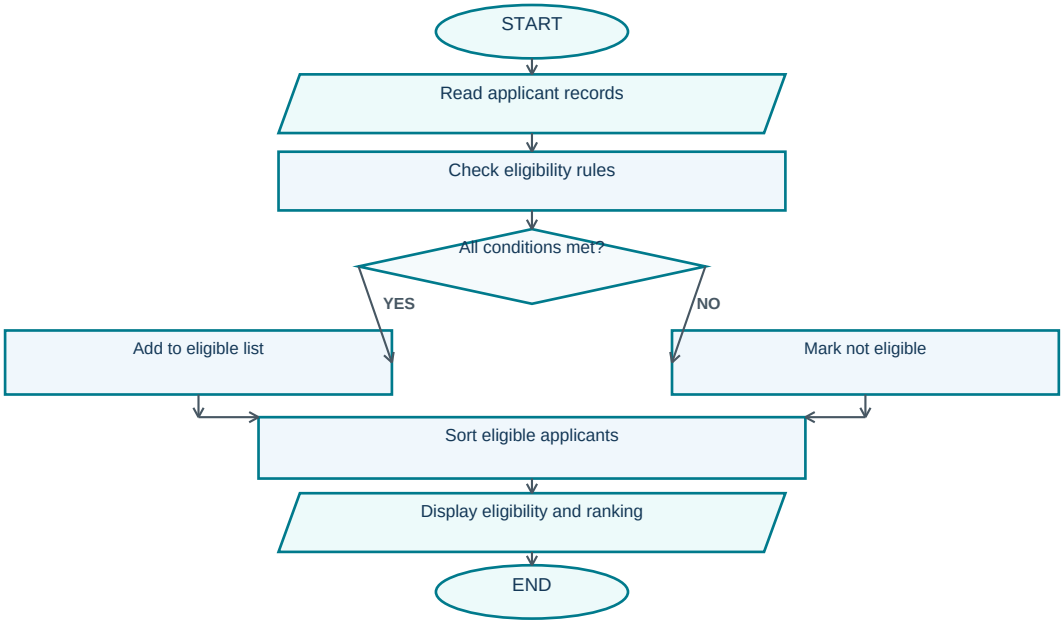
4. Add applicants who satisfy all conditions to the eligible list.

5. Sort eligible applicants by aggregate descending, attendance descending, and income ascending.

6. Print each applicant's eligibility and the ranked eligible list.

7. Stop.

5. Flowchart



CASE STUDY 1: University Scholarship Allocation — Program Design

6. Pseudocode (Line by Line)

```
BEGIN
  applicants ← sample applicant records
  eligible ← empty list
  FOR each applicant IN applicants
    IF aggregate ≥ 75 AND all subjects ≥ 60
       AND attendance ≥ 75 AND income ≤ 250000
      ADD applicant TO eligible
      SET status = Eligible
    ELSE
      SET status = Not eligible
    ENDIF
  ENDFOR
  SORT eligible by aggregate descending, attendance descending, income ascending
  DISPLAY statuses and ranked eligible applicants
END
```

7. Python Source Code (Step by Step)

The code below uses variables, conditions, loops, lists/dictionaries, and arithmetic operations where appropriate. Read the comments and output labels to follow the execution.

```
applicants = [
    {"name": "Asha", "aggregate": 91, "subjects": [88, 94, 91],
     "attendance": 92, "income": 180000},
    {"name": "Bharat", "aggregate": 84, "subjects": [78, 82, 92],
     "attendance": 88, "income": 220000},
    {"name": "Charan", "aggregate": 79, "subjects": [58, 86, 93],
     "attendance": 90, "income": 150000},
    {"name": "Divya", "aggregate": 84, "subjects": [80, 84, 88],
     "attendance": 95, "income": 190000},
]

eligible = []
print("UNIVERSITY SCHOLARSHIP ALLOCATION")
for student in applicants:
    meets_rules = (
        student["aggregate"] >= 75
        and min(student["subjects"]) >= 60
        and student["attendance"] >= 75
        and student["income"] <= 250000
    )
    status = "Eligible" if meets_rules else "Not eligible"
    print(f'{student["name"]}: {status}')
    if meets_rules:
        eligible.append(student)

eligible.sort(key=lambda s: (-s["aggregate"],
                             -s["attendance"], s["income"]))
print("\nMERIT RANK LIST")
for rank, student in enumerate(eligible, start=1):
    print(f'{rank}. {student["name"]} - {student["aggregate"]}%')
```

CASE STUDY 1: University Scholarship Allocation — Results

8. Sample Output Screenshot



9. Test Cases

Test input / condition	Expected result
Asha: all rules satisfied	Eligible; rank 1
Charan: subject mark 58%	Not eligible
Divya and Bharat: same aggregate	Divya ranks higher due to attendance

10. Simple Output Summary

```
UNIVERSITY SCHOLARSHIP ALLOCATION
Asha: Eligible
Bharat: Eligible
Charan: Not eligible
Divya: Eligible

MERIT RANK LIST
1. Asha - 91%
2. Divya - 84%
3. Bharat - 84%
```

11. Conclusion

The program applies scholarship rules consistently and produces a transparent merit list. In a real system, criteria and applicant records should be verified by the university.

CASE STUDY 2: Progressive Electricity Billing

1. Problem Statement

Create an electricity bill calculator that charges different rates for different consumption slabs. The program must calculate the energy charge, fixed charge, and final bill from the number of units used.

2. Objectives

• Accept electricity units as input.

• Apply slab-wise rates without charging all units at the highest rate.

• Show a clear bill breakdown.

3. Assumptions and Rules

• Sample domestic tariff: first 100 units at ■1.50/unit; next 100 at ■2.50/unit; units above 200 at ■4.00/unit.

• Fixed charge is ■50.

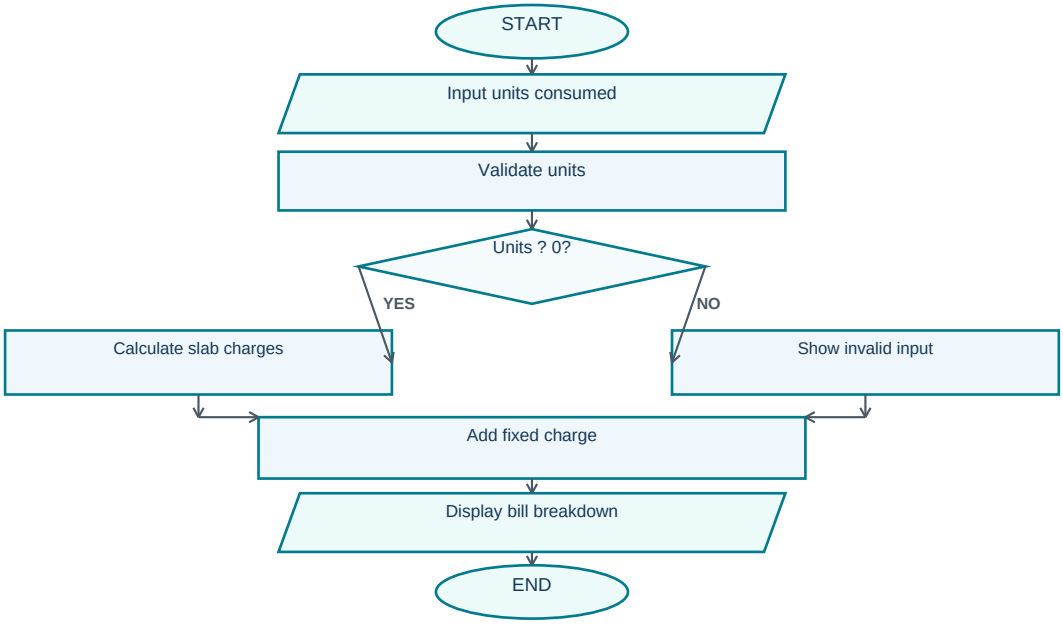
• This is an educational example, not an official utility tariff.

• Units cannot be negative.

4. Algorithm (Step by Step)

1. Start.
2. Read units consumed.
3. If units are negative, show an invalid-input message.
4. Calculate the first 100 units at ■1.50 per unit.
5. Calculate units 101–200 at ■2.50 per unit.
6. Calculate units above 200 at ■4.00 per unit.
7. Add the fixed charge of ■50.
8. Display slab charges and total bill.
9. Stop.

5. Flowchart



CASE STUDY 2: Progressive Electricity Billing — Program Design

6. Pseudocode (Line by Line)

```
BEGIN
  READ units
  IF units < 0 THEN
    DISPLAY invalid input
  ELSE
    charge ← 0
    charge ← charge + min(units,100) × 1.50
    charge ← charge + min(max(units-100,0),100) × 2.50
    charge ← charge + max(units-200,0) × 4.00
    total ← charge + 50
    DISPLAY charge, fixed charge, total
  ENDIF
END
```

7. Python Source Code (Step by Step)

The code below uses variables, conditions, loops, lists/dictionaries, and arithmetic operations where appropriate. Read the comments and output labels to follow the execution.

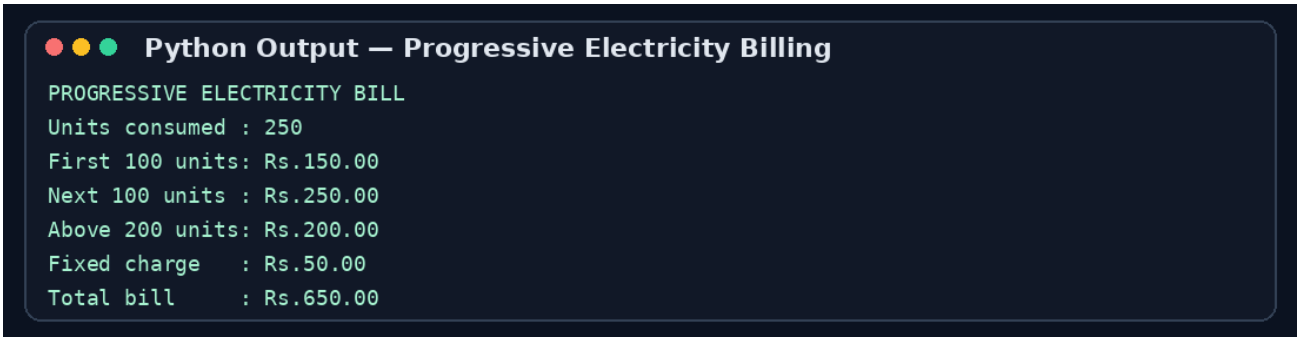
```
units = 250
fixed_charge = 50.00

if units < 0:
    print("Invalid units")
else:
    first_slab = min(units, 100) * 1.50
    second_slab = min(max(units - 100, 0), 100) * 2.50
    third_slab = max(units - 200, 0) * 4.00
    energy_charge = first_slab + second_slab + third_slab
    total = energy_charge + fixed_charge

    print("PROGRESSIVE ELECTRICITY BILL")
    print(f"Units consumed : {units}")
    print(f"First 100 units: Rs.{first_slab:.2f}")
    print(f"Next 100 units : Rs.{second_slab:.2f}")
    print(f"Above 200 units: Rs.{third_slab:.2f}")
    print(f"Fixed charge   : Rs.{fixed_charge:.2f}")
    print(f"Total bill     : Rs.{total:.2f}")
```

CASE STUDY 2: Progressive Electricity Billing — Results

8. Sample Output Screenshot



9. Test Cases

Test input / condition	Expected result
0 units	Total = ■50.00 fixed charge only
100 units	Energy ■150.00; total ■200.00
250 units	Energy ■600.00; total ■650.00
-5 units	Invalid units

10. Simple Output Summary

```
PROGRESSIVE ELECTRICITY BILL
Units consumed : 250
First 100 units: Rs.150.00
Next 100 units : Rs.250.00
Above 200 units: Rs.200.00
Fixed charge   : Rs.50.00
Total bill     : Rs.650.00
```

11. Conclusion

The program demonstrates progressive slab billing and makes the calculation easy to audit. Actual bills must use the current tariff published by the electricity provider.

CASE STUDY 3: Hospital Billing and Patient Classification

1. Problem Statement

Prepare a hospital bill based on room category, number of days, consultation fee, and tests. Classify the patient as outpatient or inpatient and display the itemized amount.

2. Objectives

• Calculate room and service charges accurately.

• Classify patients from their admission status.

• Display a readable itemized bill.

3. Assumptions and Rules

• General room costs ■1,000/day; private room costs ■2,500/day.

• Consultation fee is ■500; tests cost ■800 in this sample.

• Outpatients have zero room days; inpatients have at least one room day.

• All amounts are sample educational rates.

4. Algorithm (Step by Step)

1. Start.

2. Read patient name, patient type, room type, and stay days.

3. Set room rate based on room type.

4. If outpatient, set room charge to zero; otherwise multiply rate by days.

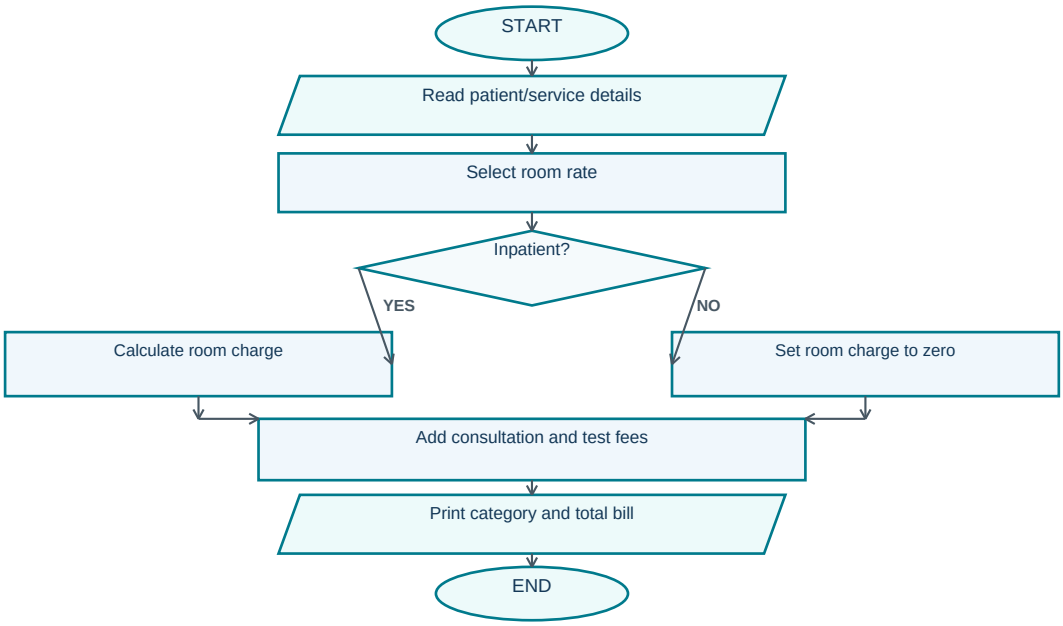
5. Add consultation and test charges.

6. Calculate total bill.

7. Display patient classification and itemized charges.

8. Stop.

5. Flowchart



CASE STUDY 3: Hospital Billing and Patient Classification — Program Design

6. Pseudocode (Line by Line)

```
BEGIN
  READ name, patient_type, room_type, days
  IF room_type = general THEN rate ← 1000
  ELSE rate ← 2500
  ENDIF
  IF patient_type = outpatient THEN room_charge ← 0
  ELSE room_charge ← rate × days
  ENDIF
  total ← room_charge + 500 + 800
  DISPLAY patient_type, room_charge, consultation, tests, total
END
```

7. Python Source Code (Step by Step)

The code below uses variables, conditions, loops, lists/dictionaries, and arithmetic operations where appropriate. Read the comments and output labels to follow the execution.

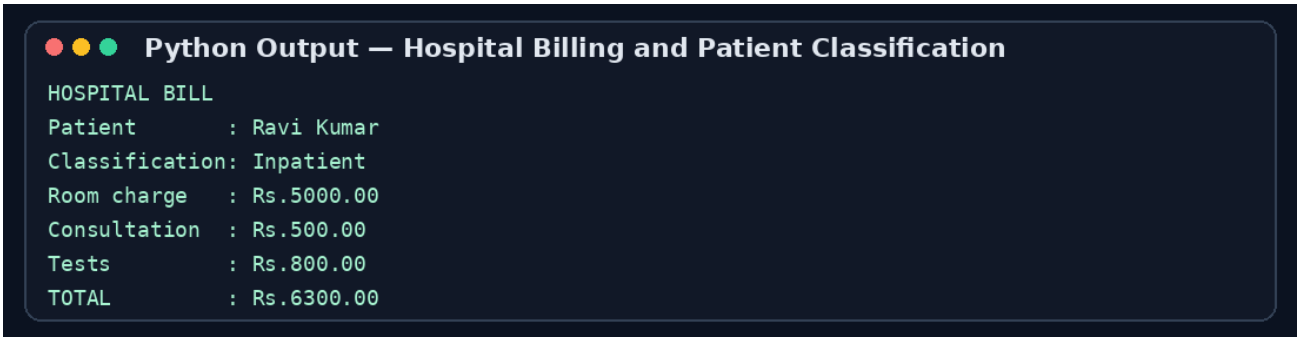
```
patient_name = "Ravi Kumar"
patient_type = "inpatient"
room_type = "private"
stay_days = 2
consultation_fee = 500.00
test_charges = 800.00

room_rates = {"general": 1000.00, "private": 2500.00}
if patient_type == "outpatient":
    room_charge = 0.00
elif patient_type == "inpatient" and stay_days >= 1:
    room_charge = room_rates[room_type] * stay_days
else:
    room_charge = 0.00
    print("Check patient type or stay days")

total = room_charge + consultation_fee + test_charges
print("HOSPITAL BILL")
print(f"Patient       : {patient_name}")
print(f"Classification: {patient_type.title()}")
print(f"Room charge   : Rs.{room_charge:.2f}")
print(f"Consultation  : Rs.{consultation_fee:.2f}")
print(f"Tests         : Rs.{test_charges:.2f}")
print(f"TOTAL         : Rs.{total:.2f}")
```

CASE STUDY 3: Hospital Billing and Patient Classification — Results

8. Sample Output Screenshot



9. Test Cases

Test input / condition	Expected result
Inpatient, private room, 2 days	Total ■6,300
Outpatient	Room charge ■0
General room, 1 day	Room charge ■1,000

10. Simple Output Summary

```
HOSPITAL BILL
Patient       : Ravi Kumar
Classification: Inpatient
Room charge   : Rs.5000.00
Consultation  : Rs.500.00
Tests         : Rs.800.00
TOTAL         : Rs.6300.00
```

11. Conclusion

The example separates room, consultation, and test charges so patients can understand the bill. Real hospitals must follow their approved rates and billing policies.

CASE STUDY 4: Employee Payroll and Deduction Analysis

1. Problem Statement

Calculate an employee's gross salary, deductions, and take-home salary using basic pay, allowances, and sample deduction percentages.

2. Objectives

• Compute gross salary from pay components.

• Calculate deductions in a transparent way.

• Report net take-home salary.

3. Assumptions and Rules

• HRA is 20% of basic pay and travel allowance is 10%.

• Provident-fund deduction is 12% of basic pay; tax deduction is 5% of gross salary for this example.

• These simplified percentages are for learning and are not tax or legal advice.

4. Algorithm (Step by Step)

1. Start.

2. Read employee name and basic salary.

3. Calculate HRA and travel allowance.

4. Add basic salary and allowances to get gross salary.

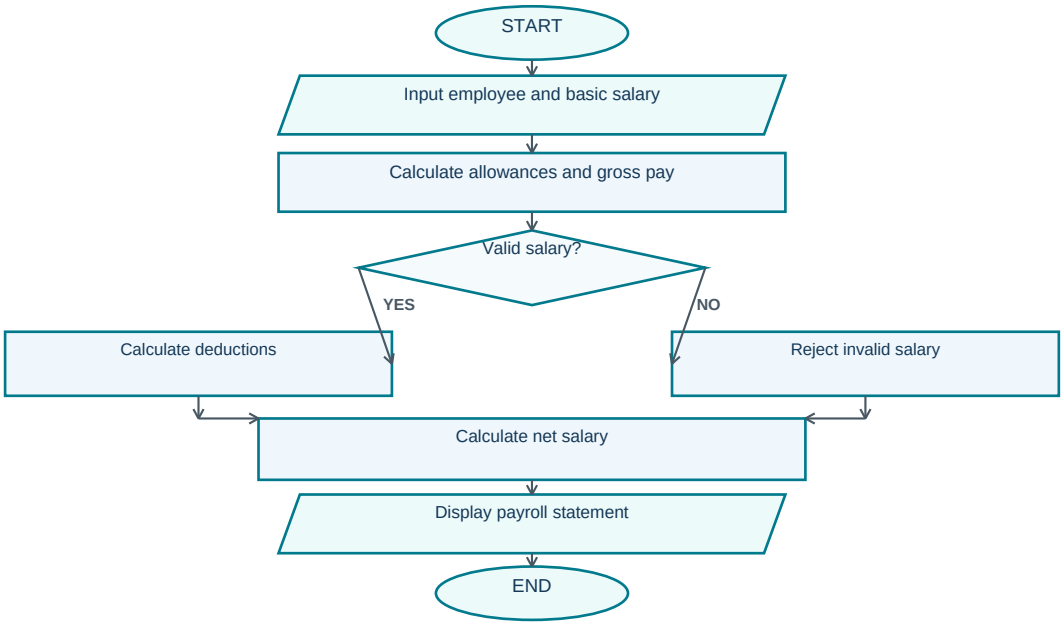
5. Calculate PF and sample tax deductions.

6. Subtract total deductions from gross salary.

7. Display salary components and take-home pay.

8. Stop.

5. Flowchart



CASE STUDY 4: Employee Payroll and Deduction Analysis — Program Design

6. Pseudocode (Line by Line)

```
BEGIN
  READ employee_name, basic_salary
  hra ← basic_salary × 0.20
  travel ← basic_salary × 0.10
  gross ← basic_salary + hra + travel
  pf ← basic_salary × 0.12
  tax ← gross × 0.05
  deductions ← pf + tax
  net ← gross - deductions
  DISPLAY salary components, deductions, net
END
```

7. Python Source Code (Step by Step)

The code below uses variables, conditions, loops, lists/dictionaries, and arithmetic operations where appropriate. Read the comments and output labels to follow the execution.

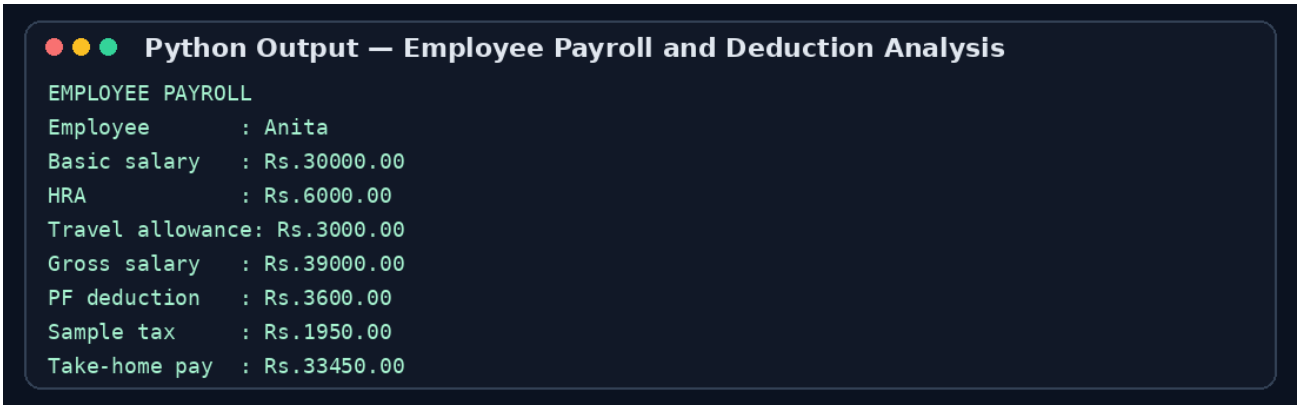
```
employee_name = "Anita"
basic_salary = 30000.00

hra = basic_salary * 0.20
travel_allowance = basic_salary * 0.10
gross_salary = basic_salary + hra + travel_allowance
pf_deduction = basic_salary * 0.12
tax_deduction = gross_salary * 0.05
total_deductions = pf_deduction + tax_deduction
net_salary = gross_salary - total_deductions

print("EMPLOYEE PAYROLL")
print(f"Employee       : {employee_name}")
print(f"Basic salary   : Rs.{basic_salary:.2f}")
print(f"HRA            : Rs.{hra:.2f}")
print(f"Travel allowance: Rs.{travel_allowance:.2f}")
print(f"Gross salary   : Rs.{gross_salary:.2f}")
print(f"PF deduction   : Rs.{pf_deduction:.2f}")
print(f"Sample tax     : Rs.{tax_deduction:.2f}")
print(f"Take-home pay  : Rs.{net_salary:.2f}")
```

CASE STUDY 4: Employee Payroll and Deduction Analysis — Results

8. Sample Output Screenshot



9. Test Cases

Test input / condition	Expected result
Basic salary ■30,000	Net salary ■33,450
Basic salary ■0	All computed values are ■0
Negative basic salary	Should be rejected in a production version

10. Simple Output Summary

```
EMPLOYEE PAYROLL
Employee       : Anita
Basic salary   : Rs.30000.00
HRA            : Rs.6000.00
Travel allowance: Rs.3000.00
Gross salary   : Rs.39000.00
PF deduction   : Rs.3600.00
Sample tax     : Rs.1950.00
Take-home pay  : Rs.33450.00
```

11. Conclusion

The payroll calculation shows how allowances and deductions affect take-home pay. A production payroll system should validate inputs and use applicable legal rules.

CASE STUDY 5: Retail Billing with Multiple Discount Rules

1. Problem Statement

Create a retail bill calculator that totals item prices, applies a discount based on purchase value, and then applies a member discount where permitted.

2. Objectives

• Calculate subtotal from item quantities and prices.

• Apply discount rules in a defined order.

• Show savings and the final amount payable.

3. Assumptions and Rules

• Subtotal of ■5,000 or more receives 10% discount; ■2,000–■4,999.99 receives 5%; otherwise no purchase discount.

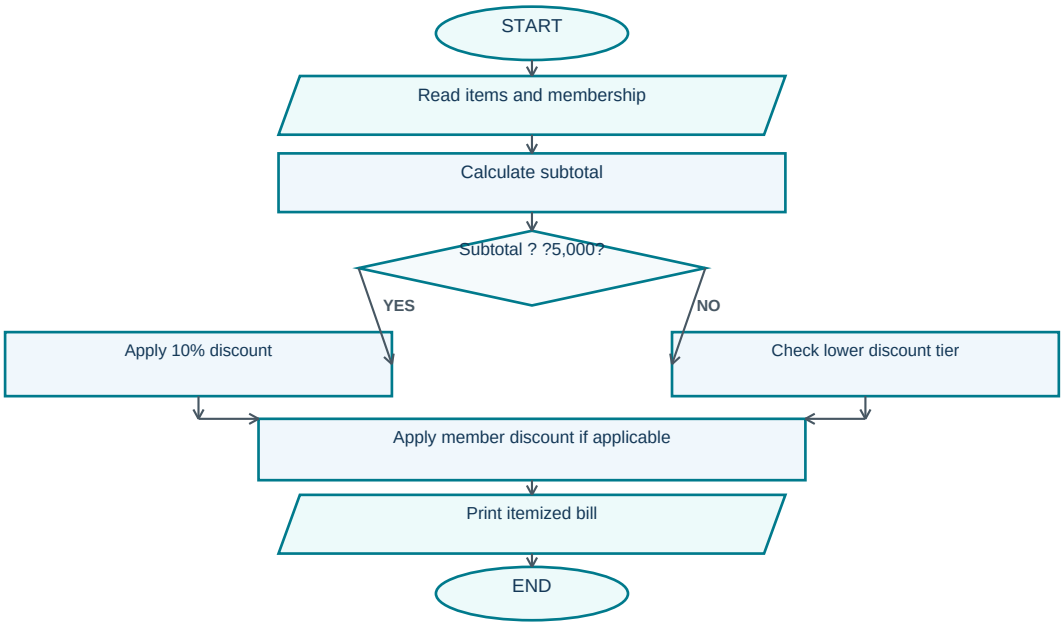
• Members receive an additional 2% discount after the purchase discount.

• Taxes are excluded to keep the example focused.

4. Algorithm (Step by Step)

1. Start.
2. Read item prices and quantities; calculate each line total.
3. Add line totals to obtain subtotal.
4. Choose purchase discount based on subtotal.
5. Calculate discounted amount.
6. If the customer is a member, apply 2% member discount to the discounted amount.
7. Calculate final payable amount.
8. Display subtotal, discounts, and final amount.
9. Stop.

5. Flowchart



CASE STUDY 5: Retail Billing with Multiple Discount Rules — Program Design

6. Pseudocode (Line by Line)

```
BEGIN
  READ item records and member_status
  subtotal ← SUM(price × quantity)
  IF subtotal ≥ 5000 THEN rate ← 0.10
  ELSE IF subtotal ≥ 2000 THEN rate ← 0.05
  ELSE rate ← 0
  ENDIF
  purchase_discount ← subtotal × rate
  after_discount ← subtotal - purchase_discount
  IF member_status THEN member_discount ← after_discount × 0.02
  ELSE member_discount ← 0
  ENDIF
  final_amount ← after_discount - member_discount
  DISPLAY subtotal, discounts, final_amount
END
```

7. Python Source Code (Step by Step)

The code below uses variables, conditions, loops, lists/dictionaries, and arithmetic operations where appropriate. Read the comments and output labels to follow the execution.

```
items = [
    {"name": "Notebook", "price": 80.00, "quantity": 10},
    {"name": "Pen pack", "price": 120.00, "quantity": 5},
    {"name": "Bag", "price": 2500.00, "quantity": 1},
]
is_member = True

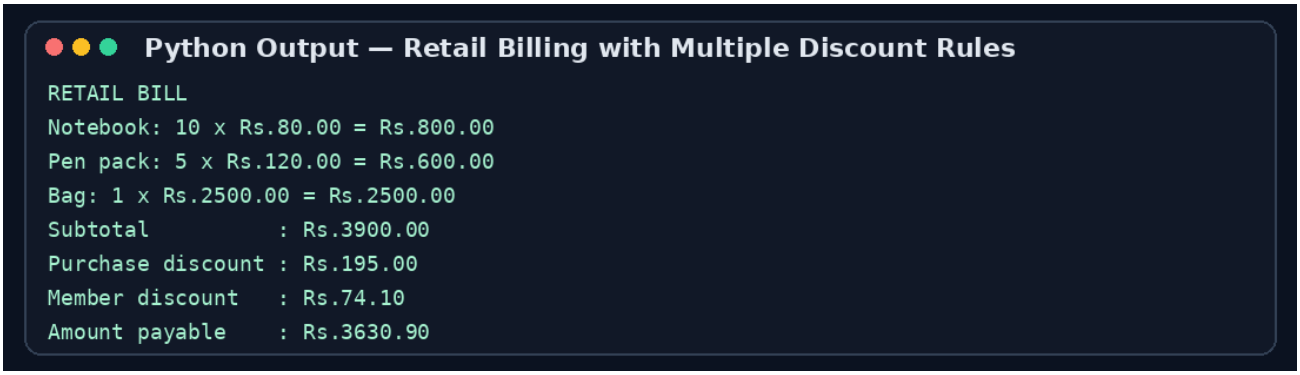
subtotal = sum(item["price"] * item["quantity"] for item in items)
if subtotal >= 5000:
    purchase_rate = 0.10
elif subtotal >= 2000:
    purchase_rate = 0.05
else:
    purchase_rate = 0.00

purchase_discount = subtotal * purchase_rate
after_discount = subtotal - purchase_discount
member_discount = after_discount * 0.02 if is_member else 0.00
final_amount = after_discount - member_discount

print("RETAIL BILL")
for item in items:
    line_total = item["price"] * item["quantity"]
    print(f'{item["name"]}: {item["quantity"]} x '
          f'Rs.{item["price"]:.2f} = Rs.{line_total:.2f}')
print(f"Subtotal          : Rs.{subtotal:.2f}")
print(f"Purchase discount : Rs.{purchase_discount:.2f}")
print(f"Member discount   : Rs.{member_discount:.2f}")
print(f"Amount payable    : Rs.{final_amount:.2f}")
```

CASE STUDY 5: Retail Billing with Multiple Discount Rules — Results

8. Sample Output Screenshot



9. Test Cases

Test input / condition	Expected result
Subtotal ■1,000, non-member	No discount; payable ■1,000
Subtotal ■3,900, member	5% then 2% discount; payable ■3,630.90
Subtotal ■5,000, non-member	10% discount; payable ■4,500

10. Simple Output Summary

```
RETAIL BILL
Notebook: 10 x Rs.80.00 = Rs.800.00
Pen pack: 5 x Rs.120.00 = Rs.600.00
Bag: 1 x Rs.2500.00 = Rs.2500.00
Subtotal          : Rs.3900.00
Purchase discount : Rs.195.00
Member discount   : Rs.74.10
Amount payable    : Rs.3630.90
```

11. Conclusion

The bill applies discounts in a consistent order and displays each saving separately. A real store should define whether taxes are calculated before or after discounts.

CASE STUDY 6: Campus Event Financial Planning

1. Problem Statement

Estimate the total cost of a campus event, subtract sponsorship, calculate the registration fee per participant, and find the minimum number of registrations needed to cover costs.

2. Objectives

• Add all event expense categories.

• Calculate sponsorship shortfall and participant contribution.

• Estimate the break-even registration count.

3. Assumptions and Rules

• Costs include venue, equipment, catering, transport, and certificates.

• Sponsorship is subtracted from total costs before calculating the remaining amount.

• The planned participant fee is ■300.

• Minimum registrations use ceiling division; if sponsorship covers costs, the minimum is zero.

4. Algorithm (Step by Step)

1. Start.

2. Read costs, sponsorship, and registration fee.

3. Add all cost categories.

4. Calculate remaining cost after sponsorship, not below zero.

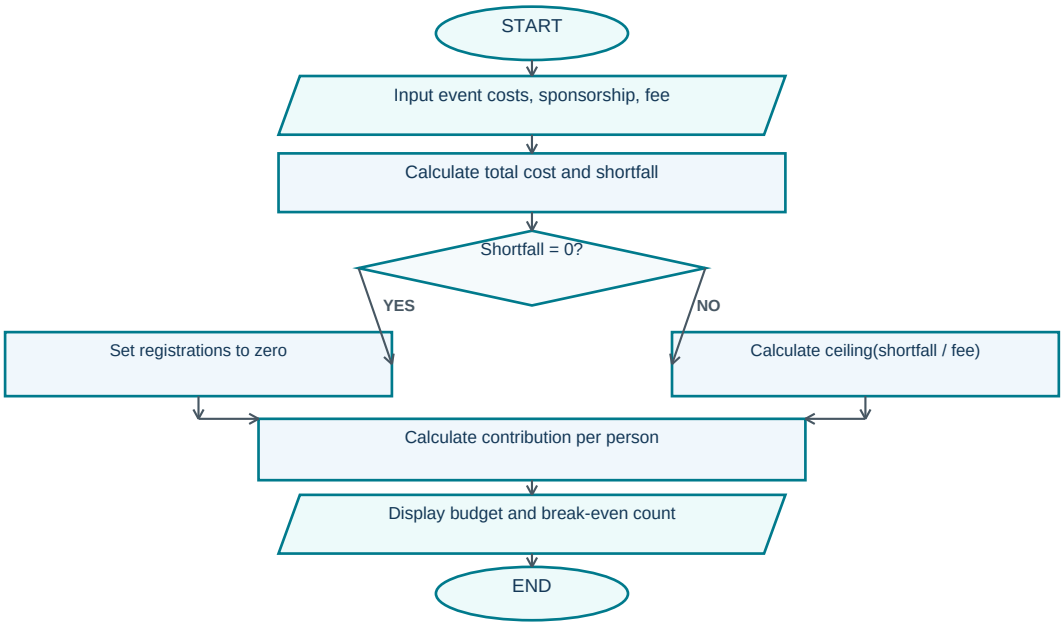
5. Calculate the required contribution per participant for the planned attendance.

6. Calculate minimum registrations using the fee and remaining cost.

7. Display total cost, shortfall, contribution, and break-even count.

8. Stop.

5. Flowchart



CASE STUDY 6: Campus Event Financial Planning — Program Design

6. Pseudocode (Line by Line)

```
BEGIN
  READ venue, equipment, catering, transport, certificates
  READ sponsorship, registration_fee, planned_people
  total_cost ← sum of all cost categories
  shortfall ← max(total_cost - sponsorship, 0)
  IF shortfall = 0 THEN minimum_people ← 0
  ELSE minimum_people ← CEILING(shortfall / registration_fee)
  ENDIF
  per_person ← shortfall / planned_people, if planned_people > 0
  DISPLAY total_cost, shortfall, per_person, minimum_people
END
```

7. Python Source Code (Step by Step)

The code below uses variables, conditions, loops, lists/dictionaries, and arithmetic operations where appropriate. Read the comments and output labels to follow the execution.

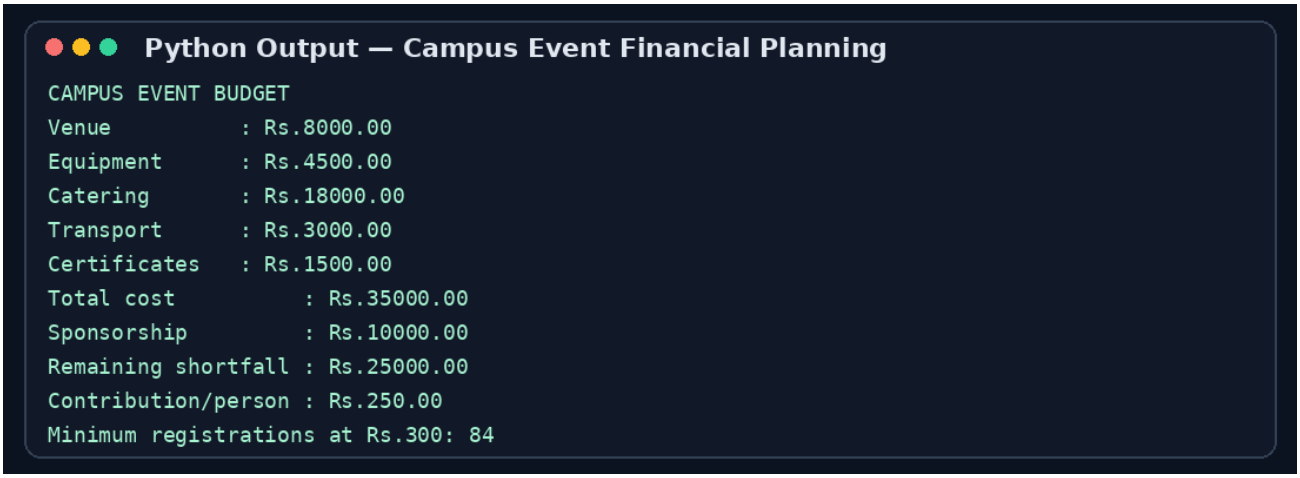
```
costs = {
    "Venue": 8000,
    "Equipment": 4500,
    "Catering": 18000,
    "Transport": 3000,
    "Certificates": 1500,
}
sponsorship = 10000
registration_fee = 300
planned_people = 100

total_cost = sum(costs.values())
shortfall = max(total_cost - sponsorship, 0)
minimum_registrations = (
    0 if shortfall == 0
    else (shortfall + registration_fee - 1) // registration_fee
)
per_person = shortfall / planned_people if planned_people > 0 else 0

print("CAMPUS EVENT BUDGET")
for name, amount in costs.items():
    print(f"{name:15}: Rs.{amount:.2f}")
print(f"Total cost          : Rs.{total_cost:.2f}")
print(f"Sponsorship         : Rs.{sponsorship:.2f}")
print(f"Remaining shortfall : Rs.{shortfall:.2f}")
print(f"Contribution/person : Rs.{per_person:.2f}")
print(f"Minimum registrations at Rs.{registration_fee}: "
      f"{minimum_registrations}")
```

CASE STUDY 6: Campus Event Financial Planning — Results

8. Sample Output Screenshot



9. Test Cases

Test input / condition	Expected result
Total ■35,000; sponsorship ■10,000	Shortfall ■25,000
Fee ■300	Minimum 84 registrations
Sponsorship greater than total cost	Shortfall and minimum registrations are zero

10. Simple Output Summary

```
CAMPUS EVENT BUDGET
Venue          : Rs.8000.00
Equipment      : Rs.4500.00
Catering       : Rs.18000.00
Transport      : Rs.3000.00
Certificates   : Rs.1500.00
Total cost          : Rs.35000.00
Sponsorship         : Rs.10000.00
Remaining shortfall : Rs.25000.00
Contribution/person : Rs.250.00
Minimum registrations at Rs.300: 84
```

11. Conclusion

The budget model helps organizers estimate event affordability before spending. Actual costs should be updated with supplier quotations and confirmed sponsorship.

CASE STUDY 7: Attendance Recovery Computation

1. Problem Statement

Calculate a student's attendance percentage and determine how many additional classes must be attended to reach a required attendance percentage, assuming no further absences.

2. Objectives

• Calculate current attendance percentage.

• Check whether the student meets the minimum requirement.

• Compute the number of consecutive classes needed to reach the target.

3. Assumptions and Rules

• Attended classes cannot exceed conducted classes.

• Minimum required attendance is 75%.

• Additional classes are attended without any absence.

• If attendance is already sufficient, the required number is zero.

4. Algorithm (Step by Step)

1. Start.

2. Read attended classes and total conducted classes.

3. Validate that counts are non-negative and attended does not exceed total.

4. Calculate current attendance percentage.

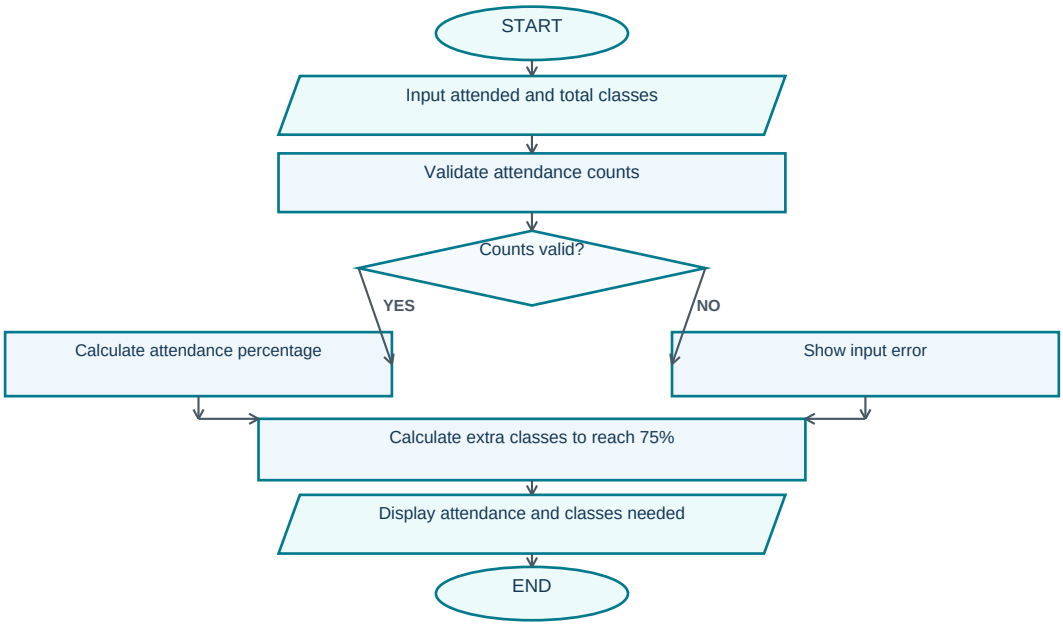
5. If percentage is at least 75%, required additional classes are zero.

6. Otherwise solve (attended + x)/(total + x) ≥ 0.75 for the smallest whole number x.

7. Display current percentage and additional classes required.

8. Stop.

5. Flowchart



CASE STUDY 7: Attendance Recovery Computation — Program Design

6. Pseudocode (Line by Line)

```
BEGIN
  READ attended, total
  IF attended < 0 OR total < 0 OR attended > total THEN
    DISPLAY invalid input
  ELSE IF total = 0 THEN DISPLAY no classes conducted
  ELSE
    percent ← attended / total × 100
    IF percent ≥ 75 THEN extra ← 0
    ELSE extra ← CEILING((0.75 × total - attended) / 0.25)
    ENDIF
    DISPLAY percent and extra
  ENDIF
END
```

7. Python Source Code (Step by Step)

The code below uses variables, conditions, loops, lists/dictionaries, and arithmetic operations where appropriate. Read the comments and output labels to follow the execution.

```
attended = 42
total_classes = 60
required_percent = 75

if attended < 0 or total_classes < 0 or attended > total_classes:
    print("Invalid attendance counts")
elif total_classes == 0:
    print("No classes have been conducted yet")
else:
    percentage = attended / total_classes * 100
    if percentage >= required_percent:
        extra_classes = 0
    else:
        # Solve (attended + x) / (total_classes + x) >= 0.75
        extra_classes = int(
            (required_percent * total_classes / 100 - attended)
            / (1 - required_percent / 100)
        )
        if (attended + extra_classes) / (total_classes + extra_classes) * 100 < required_percent:
            extra_classes += 1

    print("ATTENDANCE RECOVERY")
    print(f"Classes attended : {attended}")
    print(f"Classes conducted: {total_classes}")
    print(f"Current attendance: {percentage:.2f}%")
    print(f"Extra classes needed: {extra_classes}")
```

CASE STUDY 7: Attendance Recovery Computation — Results

8. Sample Output Screenshot



9. Test Cases

Test input / condition	Expected result
42/60 classes	70%; 12 extra classes to reach 75%
45/60 classes	75%; no extra classes needed
Attended 10, total 8	Invalid attendance counts

10. Simple Output Summary

```
ATTENDANCE RECOVERY
Classes attended : 42
Classes conducted: 60
Current attendance: 70.00%
Extra classes needed: 12
```

11. Conclusion

The program estimates recovery classes using a mathematical condition rather than simply adding a percentage to the current total. Students should follow their institution's official attendance policy.

CASE STUDY 8: ATM Transaction Sequence Simulation

1. Problem Statement

Simulate a sequence of deposits and withdrawals while maintaining the account balance. A withdrawal must be rejected if the amount is invalid or the balance is insufficient.

2. Objectives

• Process transactions in order.

• Prevent negative balances and invalid transaction amounts.

• Show the result of each transaction and final balance.

3. Assumptions and Rules

• Opening balance is ■5,000.

• Amounts must be positive.

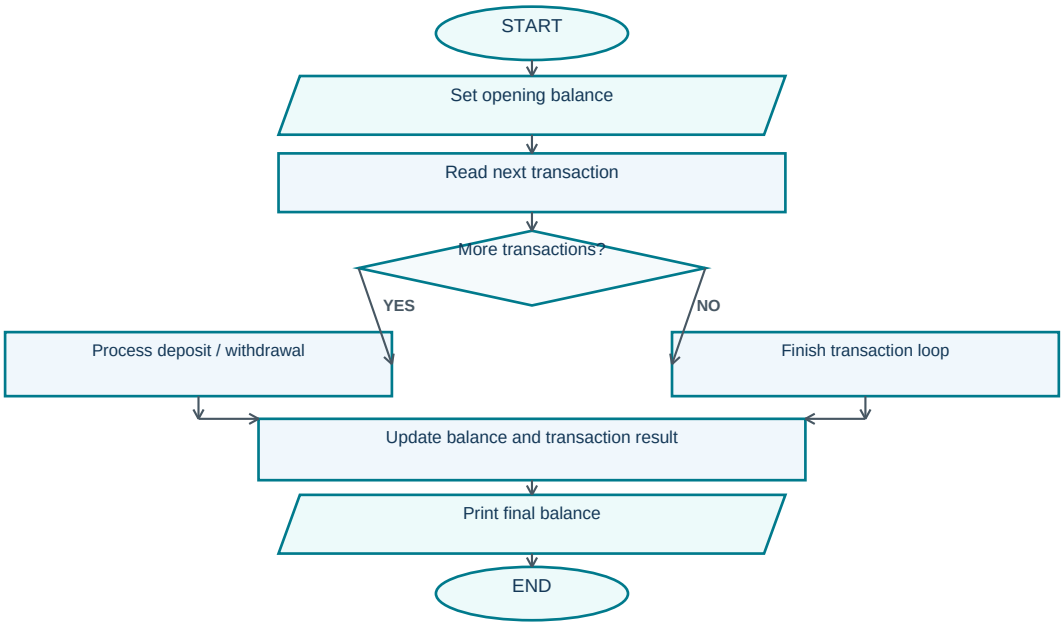
• Withdrawals cannot exceed the current balance.

• The example uses a fixed sequence rather than connecting to a bank.

4. Algorithm (Step by Step)

1. Start with the opening balance.
2. Read each transaction type and amount in sequence.
3. For a deposit, add a positive amount to the balance.
4. For a withdrawal, check amount and available balance.
5. Reject invalid or unsupported transactions without changing the balance.
6. Print the result and balance after each transaction.
7. Print final balance.
8. Stop.

5. Flowchart



CASE STUDY 8: ATM Transaction Sequence Simulation — Program Design

6. Pseudocode (Line by Line)

```
BEGIN
  balance ← 5000
  transactions ← [(deposit, 2000), (withdraw, 1500), (withdraw, 7000)]
  FOR each transaction
    IF amount ≤ 0 THEN DISPLAY invalid amount
    ELSE IF type = deposit THEN balance ← balance + amount
    ELSE IF type = withdrawal AND amount ≤ balance
      balance ← balance - amount
    ELSE IF type = withdrawal DISPLAY insufficient funds
    ELSE DISPLAY unknown transaction
    ENDIF
    DISPLAY transaction result and balance
  ENDFOR
  DISPLAY final balance
END
```

7. Python Source Code (Step by Step)

The code below uses variables, conditions, loops, lists/dictionaries, and arithmetic operations where appropriate. Read the comments and output labels to follow the execution.

```
balance = 5000.00
transactions = [
    ("deposit", 2000.00),
    ("withdraw", 1500.00),
    ("withdraw", 7000.00),
]

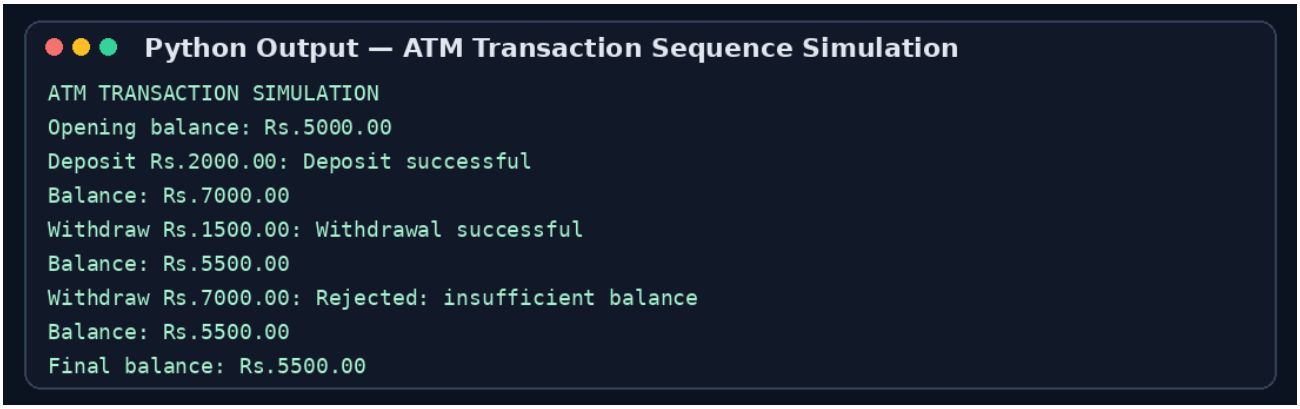
print("ATM TRANSACTION SIMULATION")
print(f"Opening balance: Rs.{balance:.2f}")
for transaction_type, amount in transactions:
    if amount <= 0:
        result = "Rejected: amount must be positive"
    elif transaction_type == "deposit":
        balance += amount
        result = "Deposit successful"
    elif transaction_type == "withdraw":
        if amount <= balance:
            balance -= amount
            result = "Withdrawal successful"
        else:
            result = "Rejected: insufficient balance"
    else:
        result = "Rejected: unknown transaction"

    print(f"{transaction_type.title()} Rs.{amount:.2f}: {result}")
    print(f"Balance: Rs.{balance:.2f}")

print(f"Final balance: Rs.{balance:.2f}")
```

CASE STUDY 8: ATM Transaction Sequence Simulation — Results

8. Sample Output Screenshot



9. Test Cases

Test input / condition	Expected result
Deposit ■2,000 into ■5,000	Balance becomes ■7,000
Withdraw ■1,500	Balance becomes ■5,500
Withdraw ■7,000 from ■5,500	Rejected; balance remains ■5,500
Withdraw ■0	Rejected as invalid amount

10. Simple Output Summary

```
ATM TRANSACTION SIMULATION
Opening balance: Rs.5000.00
Deposit Rs.2000.00: Deposit successful
Balance: Rs.7000.00
Withdraw Rs.1500.00: Withdrawal successful
Balance: Rs.5500.00
Withdraw Rs.7000.00: Rejected: insufficient balance
Balance: Rs.5500.00
Final balance: Rs.5500.00
```

11. Conclusion

The simulation demonstrates sequential state updates and basic transaction validation. Real banking systems require authentication, encryption, audit logs, and secure transaction handling.

CASE STUDY 9: Warehouse Inventory Reconciliation

1. Problem Statement

Compare recorded inventory with a physical stock count, identify shortages or excess stock, and calculate the value of each difference using the item's unit cost.

2. Objectives

• Compare system stock with physical stock.

• Identify shortage, match, or excess.

• Calculate the monetary value of stock differences.

3. Assumptions and Rules

• Stock counts are whole non-negative numbers.

• Difference is physical stock minus recorded stock.

• A negative difference indicates shortage; a positive difference indicates excess.

• Unit costs are sample values.

4. Algorithm (Step by Step)

1. Start.

2. Store each product's recorded quantity, physical count, and unit cost.

3. Calculate difference = physical count – recorded quantity.

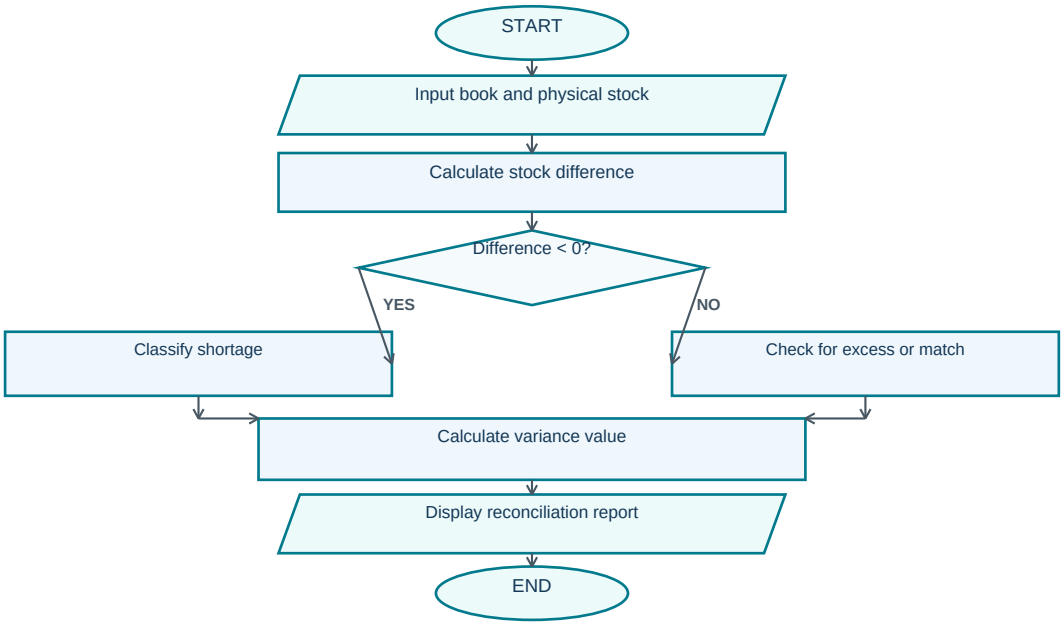
4. Classify the result as shortage, excess, or matched.

5. Multiply the absolute difference by unit cost.

6. Display each product's status and variance value.

7. Stop.

5. Flowchart



CASE STUDY 9: Warehouse Inventory Reconciliation — Program Design

6. Pseudocode (Line by Line)

```
BEGIN
  FOR each product
    difference ← physical_stock - book_stock
    IF difference < 0 THEN status ← Shortage
    ELSE IF difference > 0 THEN status ← Excess
    ELSE status ← Matched
    ENDIF
    value ← ABS(difference) × unit_cost
    DISPLAY product, difference, status, value
  ENDFOR
END
```

7. Python Source Code (Step by Step)

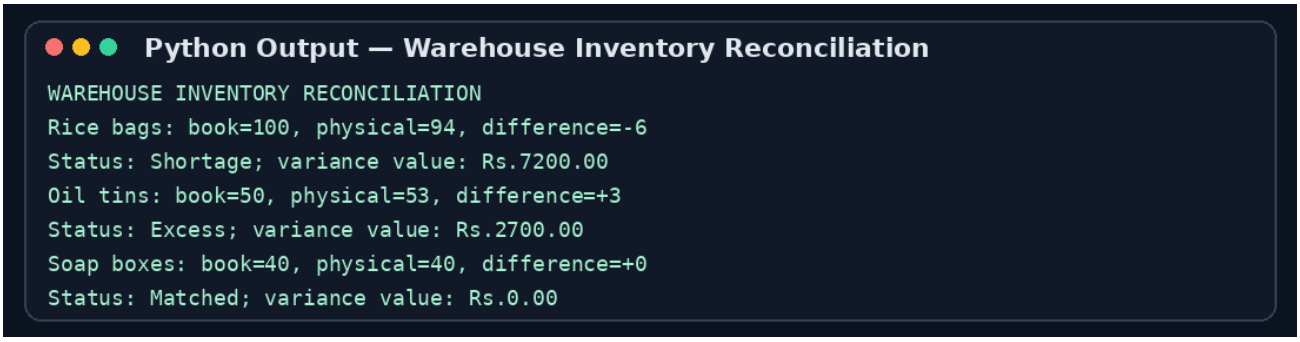
The code below uses variables, conditions, loops, lists/dictionaries, and arithmetic operations where appropriate. Read the comments and output labels to follow the execution.

```
inventory = [
    {"item": "Rice bags", "book": 100, "physical": 94, "cost": 1200},
    {"item": "Oil tins", "book": 50, "physical": 53, "cost": 900},
    {"item": "Soap boxes", "book": 40, "physical": 40, "cost": 600},
]

print("WAREHOUSE INVENTORY RECONCILIATION")
for item in inventory:
    difference = item["physical"] - item["book"]
    if difference < 0:
        status = "Shortage"
    elif difference > 0:
        status = "Excess"
    else:
        status = "Matched"
    variance_value = abs(difference) * item["cost"]
    print(f'{item["item"]}: book={item["book"]}, '
          f'physical={item["physical"]}, difference={difference:+d}')
    print(f"Status: {status}; variance value: Rs.{variance_value:.2f}")
```

CASE STUDY 9: Warehouse Inventory Reconciliation — Results

8. Sample Output Screenshot



9. Test Cases

Test input / condition	Expected result
Book 100, physical 94, cost ■1,200	Shortage 6; value ■7,200
Book 50, physical 53, cost ■900	Excess 3; value ■2,700
Book equals physical	Matched; value ■0

10. Simple Output Summary

```
WAREHOUSE INVENTORY RECONCILIATION
Rice bags: book=100, physical=94, difference=-6
Status: Shortage; variance value: Rs.7200.00
Oil tins: book=50, physical=53, difference=+3
Status: Excess; variance value: Rs.2700.00
Soap boxes: book=40, physical=40, difference=+0
Status: Matched; variance value: Rs.0.00
```

11. Conclusion

Inventory reconciliation helps identify stock discrepancies early. In a real warehouse, variances should be investigated and documented before records are adjusted.

CASE STUDY 10: Student Performance Trend Analysis

1. Problem Statement

Analyze a student's marks across multiple assessments, calculate the average and change from the first to last assessment, and describe the overall trend.

2. Objectives

• Calculate average marks and score change.

• Identify an improving, declining, or stable trend.

• Present the results in simple language.

3. Assumptions and Rules

• Marks are percentages between 0 and 100.

• At least two assessments are supplied.

• Trend is determined by comparing the last mark with the first mark; a difference within ±2 points is treated as stable.

4. Algorithm (Step by Step)

1. Start.

2. Read student name and marks from at least two assessments.

3. Validate that all marks are between 0 and 100.

4. Calculate the average mark.

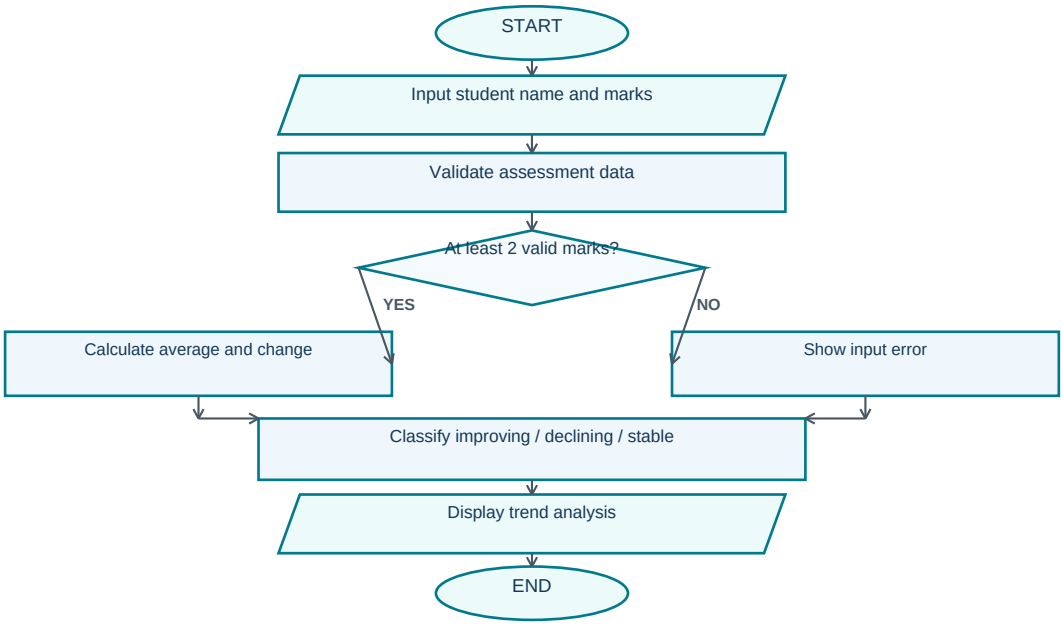
5. Calculate change = last mark – first mark.

6. If change is greater than 2, label improving; if less than –2, label declining; otherwise stable.

7. Display average, change, and trend.

8. Stop.

5. Flowchart



CASE STUDY 10: Student Performance Trend Analysis — Program Design

6. Pseudocode (Line by Line)

```
BEGIN
  READ student_name, marks
  IF length(marks) < 2 OR any mark outside 0..100 THEN
    DISPLAY invalid data
  ELSE
    average ← SUM(marks) / length(marks)
    change ← last(marks) - first(marks)
    IF change > 2 THEN trend ← Improving
    ELSE IF change < -2 THEN trend ← Declining
    ELSE trend ← Stable
    ENDIF
    DISPLAY average, change, trend
  ENDIF
END
```

7. Python Source Code (Step by Step)

The code below uses variables, conditions, loops, lists/dictionaries, and arithmetic operations where appropriate. Read the comments and output labels to follow the execution.

```
student_name = "Meena"
marks = [62, 68, 71, 79]

if len(marks) < 2 or any(mark < 0 or mark > 100 for mark in marks):
    print("Invalid assessment data")
else:
    average = sum(marks) / len(marks)
    change = marks[-1] - marks[0]
    if change > 2:
        trend = "Improving"
    elif change < -2:
        trend = "Declining"
    else:
        trend = "Stable"

    print("STUDENT PERFORMANCE TREND")
    print(f"Student: {student_name}")
    print(f"Marks: {marks}")
    print(f"Average: {average:.2f}%")
    print(f"Change from first to last: {change:+.1f} points")
    print(f"Overall trend: {trend}")
```

CASE STUDY 10: Student Performance Trend Analysis — Results

8. Sample Output Screenshot



9. Test Cases

Test input / condition	Expected result
Marks [62, 68, 71, 79]	Average 70%; improving
Marks [80, 78, 79]	Stable under ±2-point rule
Marks [90, 80, 70]	Declining
Mark 105	Invalid assessment data

10. Simple Output Summary

```
STUDENT PERFORMANCE TREND
Student: Meena
Marks: [62, 68, 71, 79]
Average: 70.00%
Change from first to last: +17.0 points
Overall trend: Improving
```

11. Conclusion

Trend analysis gives a quick summary of performance over time. It should support constructive feedback and be interpreted alongside attendance, assessment difficulty, and other learning factors.